E-INFO • FACAPE • TURNO DA MANHÃ (08H00–12H00)

Redes.tf: da Teoria à Prática com Terraform e GCP

Um mergulho 100% prático nos fundamentos de VPC, Sub-redes, CIDR, Firewalls, Cloud IAP, Cloud NAT e VPC Peering construídos como código do zero ao fim.

Facilitador: Luan Sousa — Cloud Engineer @ SantoDigital | Google Cloud Digital Leader | CC FACAPE

Quem vai guiar essa jornada e como vamos trabalhar

Facilitador: Luan Sousa

- Cloud Engineer na SantoDigital (Parceira Premier Google Cloud).
- Atuação prática em Fullstack, DevOps, Infraestrutura e SecOps.
- Certificado Google Cloud Digital Leader.
- Bacharelando em Ciência da Computação pela FACAPE.
- Foco em conectar a teoria clássica de redes com engenharia real.

Regra de Ouro: 100% Live Coding

- Zero ClickOps: não criamos recursos clicando em botões no painel.
- Construção do zero ao fim em terraform/live-coding/.
- Checkpoints prontos em terraform/labs/ caso alguém atrase.
- Evolução de recursos nativos (Labs 01–04) até Módulos (Lab 05).

Nossa Trilha de Construção: Manhã vs. Tarde

Manhã (08h00 – 12h00): Fundação e Segurança

- 08h00–09h00 (M1): IPv4, CIDR /24, IPs reservados e SDN Andromeda.
- 09h00–09h50 (M2): Terraform do Zero (Providers, State e Setup).
- 10h10–11h00 (M3): Lab 01 — VPC Customizada e Sub-rede Regional.
- 11h00–12h00 (M4): Lab 02 — Firewall Stateful, IAP e VM Privada.

Tarde (14h00 – 18h00): NAT, Peering e Módulos

- 14h00–15h00 (M5): Lab 03 — Roteamento, Cloud Router e Cloud NAT.
- 15h00–16h05 (M6): Lab 04 — VPC Peering com Hub + Mural Ao Vivo.
- 16h25–17h30 (M7): Lab 05 — Boas Práticas com Módulos Terraform.
- 17h30–18h00 (M8): FinOps com terraform destroy e encerramento.

Check-in do Ambiente: student_id e project_id

Por que cada aluno tem um student_id (1 a 40)?

- À tarde conectaremos todas as redes ao Hub via VPC Peering.
- Duas redes em Peering nunca podem ter o mesmo CIDR!
- Sua sub-rede é calculada como 10.<student_id>.0.0/24.
- Alunos 01–20: vpc-hub-turma-a | Alunos 21–40: vpc-hub-turma-b.
- Até 4 alunos dividem 1 projeto isolados pelo sufixo aluno-XX.

Terminal — Verificação Rápida

```
# 1. Verifique se o Terraform está instalado
terraform -version

# 2. Confira seu projeto ativo no gcloud
gcloud config get-value project

# 3. Autentique o ADC (usado pelo Terraform!)
gcloud auth application-default login

# 4. Entre na pasta onde vamos codar do zero
cd terraform/live-coding
```

Onde foram parar os cabos, switches e roteadores na Nuvem?

Rede Tradicional (On-Premises)

- Hardware dedicado: switches L2, roteadores L3 e cabos físicos.
- Mudança lenta: criar VLAN ou rota exige configurar caixas via SSH.
- Ponto único de gargalo: tráfego passa por appliance físico central.
- Escopo local: restrita a um prédio ou datacenter regional.

Google Cloud Andromeda (SDN)

- Software-Defined Networking: rotas, NAT e firewall via API.
- Virtualização no Hipervisor: firewall direto na vNIC de cada VM.
- Backbone Privado Global: cabos submarinos próprios da Google.
- Provisionamento em segundos com poucas linhas de Terraform.

Entendendo o Bloco 10.<student_id>.0.0/24 na Prática

Anatomia dos 32 Bits (4 Octetos de 8 Bits)

- 1º Octeto (10): Rede Privada RFC 1918 (8 bits fixos).
- 2º Octeto (<ID>): Seu número de aluno de 1 a 40 (+8 = 16 bits).
- 3º Octeto (0): Identificador da sub-rede principal (+8 = 24 bits).
- 4º Octeto (0..255): 8 bits livres para máquinas (2^8 = 256 IPs)!

Analogia Prática das Máscaras CIDR

- /24 (256 IPs): Os 3 primeiros números são a rua; o último é a casa.
- /16 (65.536 IPs): Fixa apenas os 2 primeiros octetos.
- /32 (1 IP exato): Trava todos os 32 bits (usado em regras de firewall).
- /0 (0.0.0.0/0): Representa qualquer IP da internet inteira!

Por que nossa 1ª VM no GCP recebe o IP .2 e não o .1?

Os 4 Endereços Reservados pelo GCP em toda Subnet

- 10.7.0.0 — Endereço de Rede (identifica a própria sub-rede).
- 10.7.0.1 — Default Gateway Virtual (roteador SDN Andromeda).
- 10.7.0.2 a .253 — 252 IPs livres para VMs (1ª VM nasce com .2!).
- 10.7.0.254 — Penúltimo IP reservado internamente pela Google.
- 10.7.0.255 — Endereço de Broadcast (padrão RFC).

Fórmula de Endereços Utilizáveis no GCP

- Em redes tradicionais perdemos 2 IPs (Rede e Broadcast).
- No Google Cloud sempre perdemos 4 IPs por sub-rede!
- Fórmula GCP: 2^(32 - CIDR) - 4.
- Para /24: 2^8 - 4 = 256 - 4 = 252 hosts utilizáveis.
- Comprovaremos o gateway 10.<ID>.0.1 no output do Lab 01!

Escopo: VPC é Global, Sub-rede é Regional, VM é Zonal

Hierarquia de Rede no Google Cloud

- 1. VPC Network (Global): atravessa o planeta inteiro sem região.
- 2. Subnetwork (Regional): fatia CIDR em uma região (us-central1).
- 3. Máquina Virtual (Zonal): vive em um datacenter (us-central1-a).
- VMs em zonas diferentes da mesma região compartilham a mesma /24!

Vantagem frente a outras Nuvens

- Uma única VPC pode ter uma sub-rede nos EUA (us-central1) e outra em São Paulo (southamerica-east1).
- As duas conversam por IP privado via fibra global da Google sem precisar configurar VPN ou Peering entre regiões!

Por que gerenciar Redes com Terraform em vez do Console?

Os Riscos do ClickOps (Painel Manual)

- Impossível replicar dev, staging e prod sem esquecer configurações.
- Regras de firewall abertas "só para teste" que nunca são removidas.
- Sem histórico de auditoria (quem mudou, quando e por quê).
- Sem revisão de código (Pull Request) antes de aplicar em produção.

O Poder do Terraform + .tfstate

- Linguagem Declarativa (HCL): você descreve o estado final desejado.
- Grafo de Dependências: cria VPC -> Subnet -> VM na ordem certa.
- Memória (.tfstate): mapeia seu código para os recursos reais no GCP.
- Previsibilidade total com terraform plan antes de aplicar mudanças.

Os 5 Blocos e os 5 Comandos do Terraform

Sintaxe de Recursos e Referência Implícita

```
resource "google_compute_network" "vpc_aluno" {
  name                    = "vpc-redes-tf-aluno-07"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "subnet" {
  name          = "subnet-us-central1-aluno-07"
  ip_cidr_range = "10.7.0.0/24"
  network       = google_compute_network.vpc_aluno.id
}
```

Os 5 Comandos Essenciais

- 1. terraform init — Baixa o provider hashicorp/google.
- 2. terraform fmt & validate — Formata e valida a sintaxe HCL.
- 3. terraform plan — Simula e mostra o diff (+ criar, ~ alterar, - destruir).
- 4. terraform apply — Provisiona no GCP e salva no .tfstate.
- 5. terraform destroy — Remove tudo na ordem reversa exata.

Preparando nossa Fundação em terraform/live-coding/

Roteiro de Construção Inicial

- 1. Criar versions.tf definindo Terraform >= 1.5 e google ~> 5.0.
- 2. Criar providers.tf apontando para var.project_id e var.region.
- 3. Criar variables.tf validando student_id entre 1 e 40.
- 4. Criar terraform.tfvars com seus dados e rodar terraform init!
- Meta antes do intervalo: "Terraform has been successfully initialized!"

terraform/live-coding/versions.tf & providers.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}
```

Modo Auto vs. Modo Custom: auto_create_subnetworks = false

VPC em Modo Automático (true) — Evite!

- Cria 1 sub-rede /20 em cada uma das +40 regiões automaticamente.
- Usa sempre o mesmo bloco fixo: 10.128.0.0/9.
- Fatal para VPC Peering: duas VPCs em modo Auto têm os mesmos IPs e nunca conseguem fazer Peering entre si!

VPC em Modo Customizado (false) — Padrão Real

- A VPC nasce 100% limpa, apenas com as sub-redes que declararmos.
- Permite planejar nosso CIDR exclusivo 10.<student_id>.0.0/24.
- Garante zero colisão de IPs com o Hub Central e VPNs.
- Habilitamos private_ip_google_access = true na sub-rede.

Construindo nossa VPC Customizada e Sub-rede Regional

Passo a Passo (Checkpoint: labs/01-vpc-subnets/)

- 1. locals.tf: calcular student_suffix e subnet_cidr (10.<ID>.0.0/24).
- 2. vpc.tf: criar google_compute_network e google_compute_subnetwork.
- 3. outputs.tf: exibir vpc_name, subnet_cidr e gateway_address.
- 4. Executar terraform plan, terraform apply e conferir no Console!

terraform/live-coding/vpc.tf

```
resource "google_compute_network" "vpc_aluno" {
  name                    = local.vpc_name
  auto_create_subnetworks = false
  routing_mode            = "REGIONAL"
}

resource "google_compute_subnetwork" "subnet_principal" {
  name                     = local.subnet_name
  ip_cidr_range            = local.subnet_cidr
  region                   = var.region
  network                  = google_compute_network.vpc_aluno.id
  private_ip_google_access = true
}
```

Firewall Stateful, Regras Implícitas e Network Tags

Regras Implícitas e Estado (Conntrack)

- Deny All Ingress (65535): bloqueia toda entrada não liberada.
- Allow All Egress (65535): permite que VMs iniciem conexões de saída.
- Stateful Firewall: se o pacote de entrada foi permitido, a resposta volta automaticamente sem precisar de regra extra!

Microsegmentação com Network Tags

- Evite aplicar regras de firewall para todas as máquinas da VPC.
- Usamos target_tags = ["ssh-iap"] na regra de firewall.
- Apenas VMs que possuam tags = ["ssh-iap"] recebem a liberação da porta 22!

Como fazer SSH em uma VM 100% Privada usando Cloud IAP?

O Perigo do IP Público na Porta 22

- Expor IP público com 0.0.0.0/0 na porta 22 atrai ataques de força bruta imediatos da internet.
- Em produção, máquinas internas e bancos nunca têm IP externo!

Solução: Cloud IAP TCP Forwarding

- Criamos a VM sem bloco access_config (IP Externo = Nenhum).
- O Cloud IAP valida sua conta Google via IAM e cria um túnel HTTPS.
- Liberamos a porta 22 apenas para o range oficial do IAP: 35.235.240.0/20.
- Comando: gcloud compute ssh ... --tunnel-through-iap.

Subindo Firewall + VM Privada e o Mistério antes do Almoço!

Passo a Passo (Checkpoint: labs/02-firewall-vms/)

- 1. firewall.tf: liberar tcp:22 p/ 35.235.240.0/20 e icmp p/ 10.0.0.0/8.
- 2. compute.tf: criar VM e2-micro Debian 12 sem access_config.
- 3. Rodar terraform apply e conectar via SSH IAP!
- Desafio das 11h50: rode ping -c 3 8.8.8.8 dentro da VM. Por que dá 100% packet loss? Resolveremos às 14h00!

terraform/live-coding/firewall.tf & compute.tf

```
resource "google_compute_firewall" "allow_ssh_iap" {
  name          = "fw-allow-ssh-iap-${local.student_suffix}"
  network       = google_compute_network.vpc_aluno.name
  direction     = "INGRESS"
  source_ranges = ["35.235.240.0/20"]
  target_tags   = ["ssh-iap"]
  allow { protocol = "tcp", ports = ["22"] }
}

resource "google_compute_instance" "vm_aluno" {
  name         = "vm-teste-${local.student_suffix}"
  machine_type = "e2-micro"
  zone         = var.zone
  tags         = ["ssh-iap"]
  boot_disk { initialize_params { image = "debian-cloud/
debian-12" } }
  network_interface {
    subnetwork = google_compute_subnetwork.subnet_principal.id
  }
}
```